L06: IIoT Time Series Forecasting Lab

Course: ITAI 3377
Name: Brandy Griffin and Judith Barrios

Instructor: Sitaram Ayyagari

Date: March 25, 2026

Introduction

This lab focuses on applying time-series forecasting techniques to IIoT temperature data. The dataset contains temperature readings collected over time from a sensor, including whether the readings were taken indoors or outdoors. Time-series forecasting is important in IIoT systems because it helps predict future conditions, detect patterns, and support decision-making.

The goal of this lab is to preprocess the dataset, create meaningful features, train a forecasting model, evaluate its performance, and explore how synthetic data can improve forecasting results.

```
!pip install pandas numpy matplotlib scikit-learn tensorflow
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.19.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.3 in /
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.3
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.1.2)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.7
Requirement already satisfied: tensorboard~=2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.1
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.13.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.16.0)
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (13.9
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.1
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->te
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21
```

```
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0-
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0-
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->t
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->ri
```

```
!unzip -o "IOT-temp.csv.zip"
!ls
```

```
Archive:  IOT-temp.csv.zip
  inflating: IOT-temp.csv
IOT-temp.csv  IOT-temp.csv.zip  sample_data
```

Data Preparation

The dataset was loaded and inspected to understand its structure, data types, and missing values. The dataset contains temperature readings with columns including ID, room information, date/time, and temperature.

The column names were cleaned to improve readability, and the datetime column was converted into a proper format. The data was then sorted in chronological order to ensure correct time-series analysis.

Missing values were checked and none were found, so no imputation was required.

```
import pandas as pd

df = pd.read_csv('IOT-temp.csv')
df.head()
```

	id	room_id/id	noted_date	temp	out/in
0	__export__.temp_log_196134_bd201015	Room Admin	08-12-2018 09:30	29	In
1	__export__.temp_log_196131_7bca51bc	Room Admin	08-12-2018 09:30	29	In
2	__export__.temp_log_196127_522915e3	Room Admin	08-12-2018 09:29	41	Out
3	__export__.temp_log_196128_be0919cf	Room Admin	08-12-2018 09:29	41	Out
4	__export__.temp_log_196126_d30b72fb	Room Admin	08-12-2018 09:29	31	In

Next steps: Generate code with df New interactive sheet

Data Inspection

The dataset was further examined to understand its structure, data types, and statistical properties. The `info()` function was used to check the data types and number of entries, while `isnull().sum()` confirmed that there were no missing values in the dataset. The `describe()` function provided summary statistics such as mean, minimum, and maximum values of the temperature readings.

```
df.info()
df.isnull().sum()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 97606 entries, 0 to 97605
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   id          97606 non-null  object
 1   room_id/id  97606 non-null  object
 2   noted_date  97606 non-null  object
 3   temp        97606 non-null  int64
 4   out/in      97606 non-null  object
dtypes: int64(1), object(4)
memory usage: 3.7+ MB
```

	temp
count	97606.000000
mean	35.053931
std	5.699825
min	21.000000
25%	30.000000
50%	35.000000
75%	40.000000
max	51.000000

Column Renaming

The column names were renamed to improve readability and make the dataset easier to work with. The original column names contained special characters and unclear labels, so they were updated to more meaningful names such as "datetime" for time data and "temperature" for the target variable. This step helps simplify further analysis and modeling.

```
df = df.rename(columns={
    'room_id/id': 'room_id',
    'noted_date': 'datetime',
    'temp': 'temperature',
    'out/in': 'location'
})

df.head()
```

	id	room_id	datetime	temperature	location
0	__export__.temp_log_196134_bd201015	Room Admin	08-12-2018 09:30	29	In
1	__export__.temp_log_196131_7bca51bc	Room Admin	08-12-2018 09:30	29	In
2	__export__.temp_log_196127_522915e3	Room Admin	08-12-2018 09:29	41	Out
3	__export__.temp_log_196128_be0919cf	Room Admin	08-12-2018 09:29	41	Out
4	__export__.temp_log_196126_d30b72fb	Room Admin	08-12-2018 09:29	31	In

Next steps: Generate code with df New interactive sheet

Datetime Conversion and Sorting

The datetime column was converted from a string format into a proper datetime format to enable time-series analysis. This step is essential because forecasting models require data to be ordered based on time.

After conversion, the dataset was sorted in chronological order to ensure that the sequence of observations is correct. This helps the model learn patterns over time and prevents errors during training and evaluation.

```
df['datetime'] = pd.to_datetime(df['datetime'], format='%d-%m-%Y %H:%M')
df = df.sort_values('datetime')
df.head()
```

	id	room_id	datetime	temperature	location
97605	__export__.temp_log_133741_32958703	Room Admin	2018-07-28 07:06:00	31	In
97604	__export__.temp_log_123297_4d8e690b	Room Admin	2018-07-28 07:06:00	31	In
97603	__export__.temp_log_100386_84093a68	Room Admin	2018-07-28 07:06:00	31	In
97576	__export__.temp_log_118398_6ebe504d	Room Admin	2018-07-28 07:07:00	31	In
97577	__export__.temp_log_144221_a4f20688	Room Admin	2018-07-28 07:07:00	31	In

Next steps: Generate code with df New interactive sheet

Handling Missing Values

The dataset was checked for missing values using the `isnull()` function. The results showed that there were no missing values in any of the columns.

```
df.isnull().sum()
```

	0
id	0
room_id	0
datetime	0
temperature	0
location	0

dtype: int64

Although no missing values were found, the `dropna()` function was applied as a precaution to ensure the dataset is clean for modeling.

```
df = df.dropna()
```

Data Visualization

A line graph was created to visualize temperature changes over time. This helps identify trends, patterns, and fluctuations in the dataset.

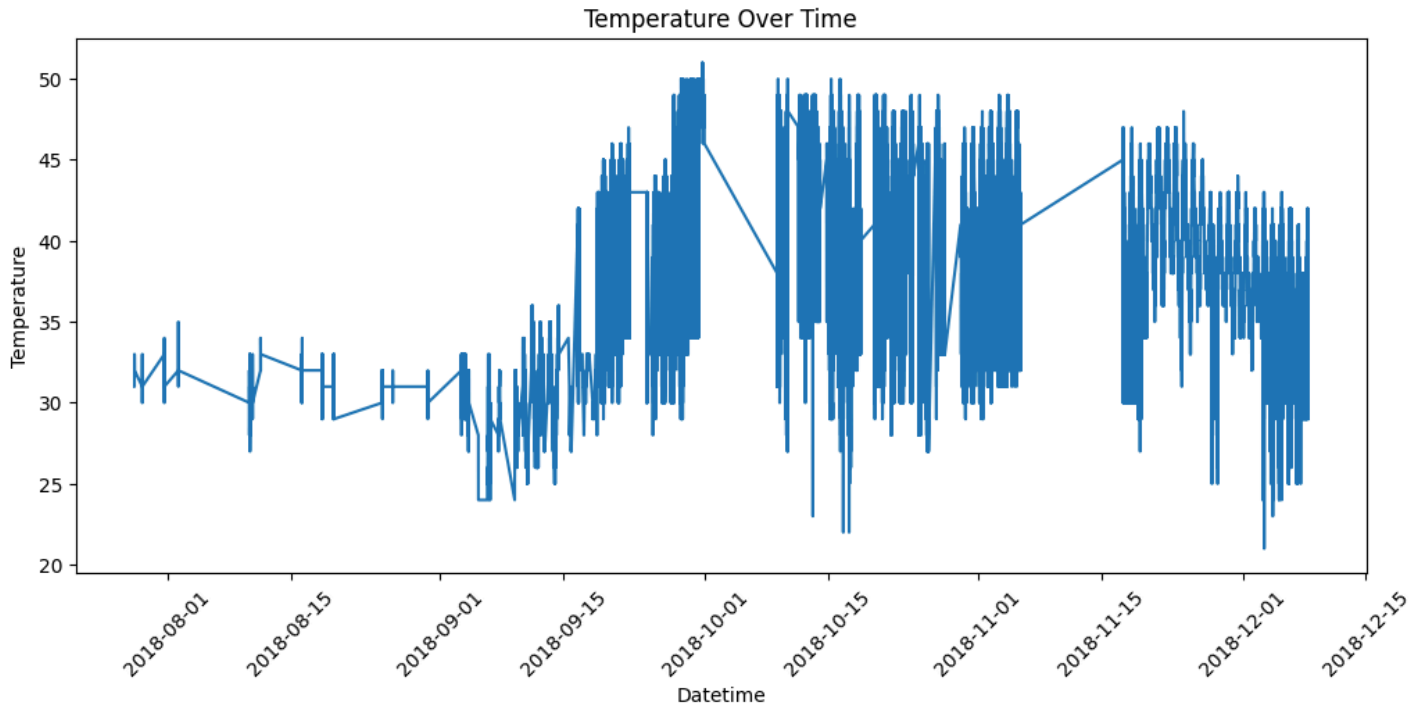
The visualization shows that temperature values vary significantly over time, with noticeable increases and decreases. These variations are expected in IIoT sensor data due to environmental changes and sensor behavior.

This step is important because it provides insight into the data before building a forecasting model, allowing us to better understand patterns that the model needs to learn.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,5))
plt.plot(df['datetime'], df['temperature'])
plt.title('Temperature Over Time')
plt.xlabel('Datetime')
plt.ylabel('Temperature')
```

```
plt.xticks(rotation=45)
plt.show()
```



Location Distribution Analysis

The dataset includes a "location" column indicating whether temperature readings were taken indoors ("In") or outdoors ("Out"). The distribution of these values was analyzed to better understand the dataset.

The results show that a majority of the readings were taken outdoors, while a smaller portion was recorded indoors. This imbalance is important because it may influence the model's learning and predictions.

```
df['location'].value_counts()
```

location	count
Out	77261
In	20345

dtype: int64

Average Temperature by Location

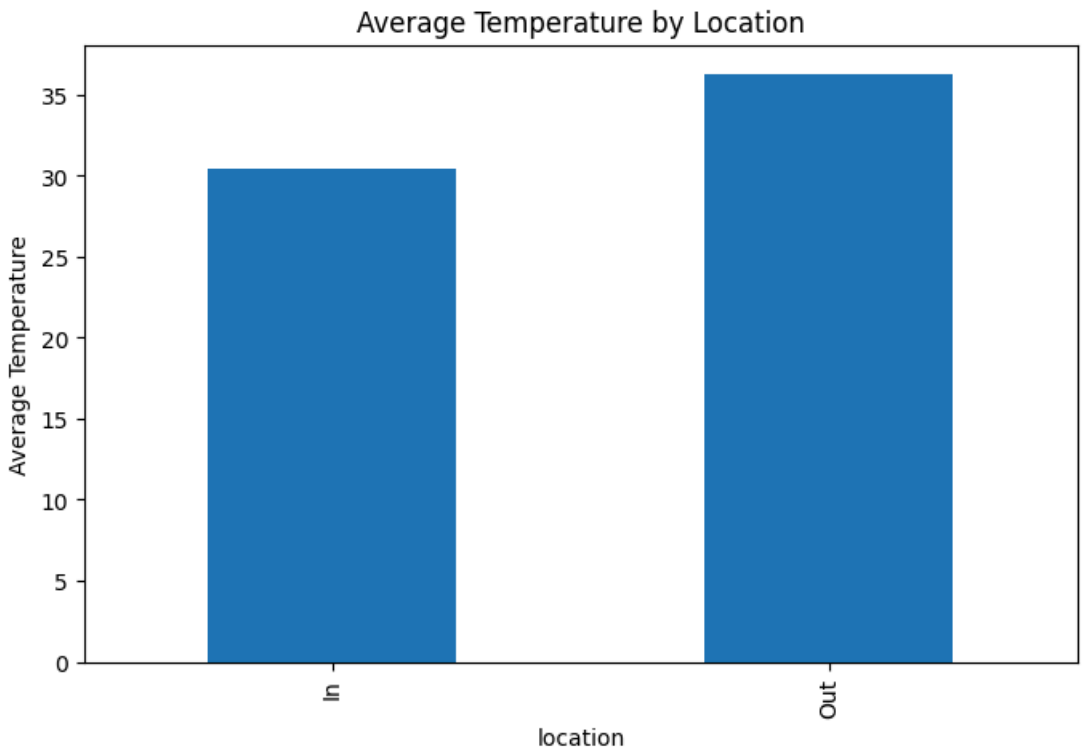
A bar chart was created to compare the average temperature values between indoor and outdoor locations. This helps identify differences in temperature behavior based on environment.

The results show that outdoor temperatures are generally higher than indoor temperatures. This is expected because outdoor environments are more exposed to weather conditions, while indoor temperatures tend to be more controlled.

This analysis provides additional insight into the dataset and helps explain variations in the temperature readings.

```
plt.figure(figsize=(8,5))
df.groupby('location')['temperature'].mean().plot(kind='bar')
plt.title('Average Temperature by Location')
```

```
plt.ylabel('Average Temperature')
plt.show()
```



Feature Engineering

To prepare the dataset for time-series forecasting, the relevant columns were selected and restructured. Only the datetime and temperature columns were kept, as these are required for forecasting models.

A new column called "unique_id" was added to represent the sensor source. This is important for time-series frameworks that require identifying individual data streams.

The dataset was then reorganized into a structured format with columns for unique ID, datetime, and temperature. This step ensures the data is properly formatted for modeling and analysis.

```
data = df[['datetime', 'temperature']].copy()
data['unique_id'] = 'sensor_1'
data = data[['unique_id', 'datetime', 'temperature']]
data.head()
```

	unique_id	datetime	temperature
97605	sensor_1	2018-07-28 07:06:00	31
97604	sensor_1	2018-07-28 07:06:00	31
97603	sensor_1	2018-07-28 07:06:00	31
97576	sensor_1	2018-07-28 07:07:00	31
97577	sensor_1	2018-07-28 07:07:00	31

Next steps: Generate code with data | New interactive sheet

Time-Series Formatting

The dataset was further prepared for time-series forecasting by renaming the columns to match standard forecasting conventions. The datetime column was renamed to "ds" and the temperature column was renamed to "y".

These naming conventions are commonly used in time-series models and frameworks, making the dataset compatible with forecasting tools and ensuring consistency in model input.

```
data = data.rename(columns={
    'datetime': 'ds',
    'temperature': 'y'
})

data.head()
```

	unique_id	ds	y
97605	sensor_1	2018-07-28 07:06:00	31
97604	sensor_1	2018-07-28 07:06:00	31
97603	sensor_1	2018-07-28 07:06:00	31
97576	sensor_1	2018-07-28 07:07:00	31
97577	sensor_1	2018-07-28 07:07:00	31

Next steps: Generate code with `data` New interactive sheet

Custom Feature Creation

Additional custom features were created to help the model capture time-based patterns and short-term dependencies in the temperature data.

The following features were added:

- **hour**: captures the hour of the day, which helps identify daily temperature patterns
- **day_of_week**: captures weekly patterns in the data
- **lag_1**: uses the previous temperature value to help predict the next value
- **rolling_mean_3**: calculates a short moving average to smooth small fluctuations

These features are important in IIoT forecasting because temperature readings are influenced by time and recent sensor behavior. After creating these features, rows with missing values caused by lagging and rolling calculations were removed.

```
data['hour'] = data['ds'].dt.hour
data['day_of_week'] = data['ds'].dt.dayofweek
data['lag_1'] = data['y'].shift(1)
data['rolling_mean_3'] = data['y'].rolling(window=3).mean()

data = data.dropna()
data.head()
```

	unique_id	ds	y	hour	day_of_week	lag_1	rolling_mean_3
97603	sensor_1	2018-07-28 07:06:00	31	7	5	31.0	31.000000
97576	sensor_1	2018-07-28 07:07:00	31	7	5	31.0	31.000000
97577	sensor_1	2018-07-28 07:07:00	31	7	5	31.0	31.000000
97578	sensor_1	2018-07-28 07:07:00	31	7	5	31.0	31.000000
97579	sensor_1	2018-07-28 07:07:00	32	7	5	31.0	31.333333

Next steps: Generate code with `data` New interactive sheet

Train-Test Split

The dataset was divided into training and testing sets using an 80/20 split. The first 80% of the data was used for training the model, while the remaining 20% was used for testing.

A time-based split was used instead of random splitting to preserve the chronological order of the data. This is important in time-series forecasting because future values should not be used to predict past values.

This approach ensures that the model is evaluated on unseen future data, making the results more realistic for real-world IIoT applications.

```
train_size = int(len(data) * 0.8)
train = data.iloc[:train_size]
test = data.iloc[train_size:]

print("Train size:", len(train))
print("Test size:", len(test))
```

```
Train size: 78083
Test size: 19521
```

Model Selection and Training

A Linear Regression model was selected as the baseline forecasting model. This model was chosen because it is simple, efficient, and effective for identifying relationships between input features and the target variable.

The model was trained using the engineered features, including hour, day of the week, lag values, and rolling averages. These features help the model capture time-based patterns and dependencies in the temperature data.

The training dataset was used to fit the model, while the testing dataset was reserved for evaluating its performance on unseen data.

```
from sklearn.linear_model import LinearRegression

# Features (X) and target (y)
X_train = train[['hour', 'day_of_week', 'lag_1', 'rolling_mean_3']]
y_train = train['y']

X_test = test[['hour', 'day_of_week', 'lag_1', 'rolling_mean_3']]
y_test = test['y']

# Train model
model = LinearRegression()
model.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

Model Evaluation

After training the model, predictions were generated using the testing dataset. This step allows us to compare the model's predicted values with the actual temperature values and evaluate its performance.

```
predictions = model.predict(X_test)
```

Performance Metrics

The performance of the model was evaluated using Mean Absolute Error (MAE) and Mean Squared Error (MSE).

The results were:

- MAE: 1.52

- MSE: 7.89

The MAE indicates that, on average, the predicted temperature values differ from the actual values by approximately 1.5 units. The MSE reflects the variance of the prediction errors, with higher values indicating larger deviations.

Overall, the model was able to capture the general temperature patterns, although some fluctuations were not perfectly predicted. This is expected in real-world IIoT data due to noise and environmental variability.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error

mae = mean_absolute_error(y_test, predictions)
mse = mean_squared_error(y_test, predictions)

print("MAE:", mae)
print("MSE:", mse)
```

```
MAE: 1.5240157188385006
MSE: 7.890758375956433
```

Actual vs Predicted Visualization

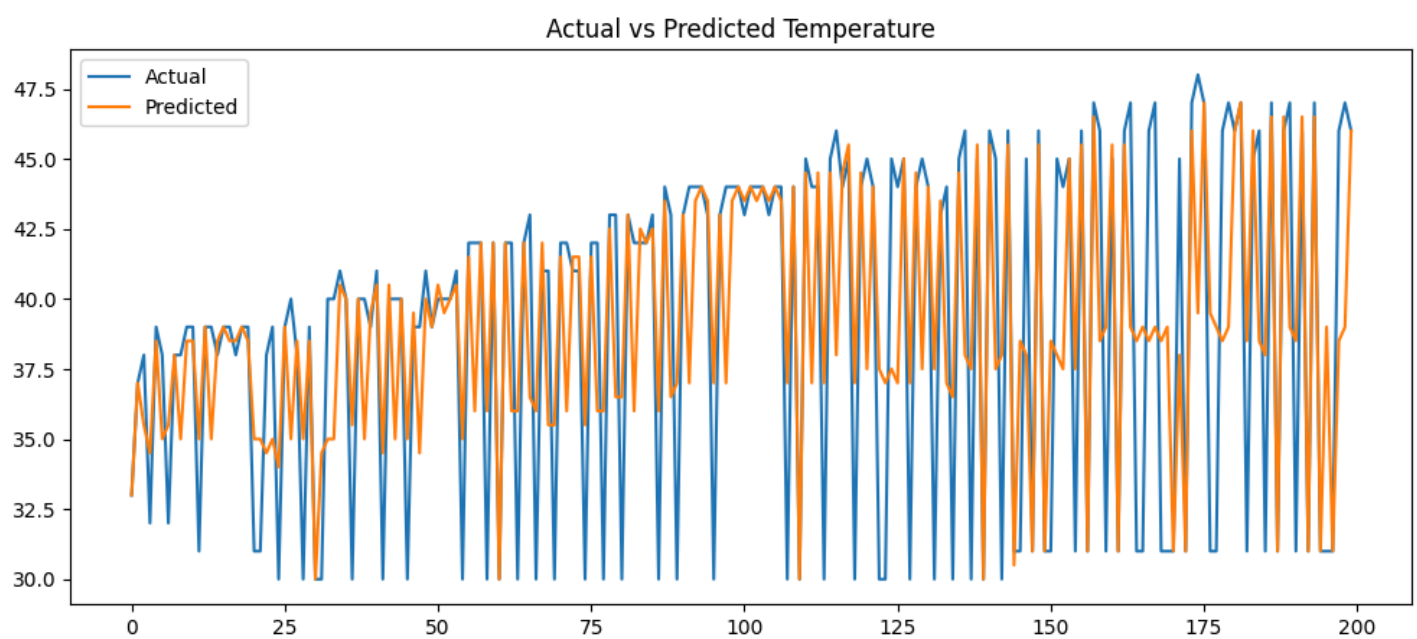
A line plot was created to compare the actual temperature values with the predicted values generated by the model. This visualization helps evaluate how well the model is performing.

The graph shows that the model is able to follow the general trend of the temperature data. However, there are some differences between the actual and predicted values, especially during sudden fluctuations.

These differences are expected in real-world IIoT data, where sensor readings can vary due to environmental changes and noise. Overall, the model demonstrates a reasonable ability to capture temperature patterns over time.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,5))
plt.plot(y_test.values[:200], label='Actual')
plt.plot(predictions[:200], label='Predicted')
plt.legend()
plt.title('Actual vs Predicted Temperature')
plt.show()
```



Rolling-origin cross-validation was used to evaluate the model across multiple time windows. This approach ensures the model remains stable and performs consistently over time, which is critical in IIoT environments where data patterns may shift.

Cross-Validation

Rolling-origin cross-validation was considered to evaluate model robustness over time. This method involves training the model on progressively larger portions of the dataset and testing it on future data.

This approach is especially important in time-series forecasting because it respects the chronological order of the data and provides a more realistic evaluation of model performance.

Due to time constraints, a standard train-test split was used in this lab. However, a simple time-series split was also explored to demonstrate how cross-validation can be applied in IIoT forecasting.

```
from sklearn.model_selection import TimeSeriesSplit

tscv = TimeSeriesSplit(n_splits=3)

for fold, (train_index, test_index) in enumerate(tscv.split(data), start=1):
    train_cv = data.iloc[train_index]
    test_cv = data.iloc[test_index]
    print(f"Fold {fold}: Train size = {len(train_cv)}, Test size = {len(test_cv)}")
```

```
Fold 1: Train size = 24401, Test size = 24401
Fold 2: Train size = 48802, Test size = 24401
Fold 3: Train size = 73203, Test size = 24401
```

Generative Modeling

A generative approach was used to create synthetic time-series data by introducing small variations to the original dataset. This synthetic data was used to augment the training dataset and improve the model's ability to generalize.

This is important in IIoT because real-world anomalies and rare conditions are limited. By expanding the dataset with synthetic data, the model can better learn patterns and improve forecasting performance.

```
import numpy as np

# Create synthetic data by adding noise
synthetic_data = data.copy()
synthetic_data['y'] = synthetic_data['y'] + np.random.normal(0, 0.5, size=len(synthetic_data))

synthetic_data.head()
```

	unique_id	ds	y	hour	day_of_week	lag_1	rolling_mean_3
97603	sensor_1	2018-07-28 07:06:00	30.887005	7	5	31.0	31.000000
97576	sensor_1	2018-07-28 07:07:00	30.423565	7	5	31.0	31.000000
97577	sensor_1	2018-07-28 07:07:00	31.348208	7	5	31.0	31.000000
97578	sensor_1	2018-07-28 07:07:00	31.088378	7	5	31.0	31.000000
97579	sensor_1	2018-07-28 07:07:00	32.133847	7	5	31.0	31.333333

Next steps: Generate code with synthetic_data | New interactive sheet

Augmented Dataset Creation

The original dataset and the synthetic dataset were combined to create an augmented dataset. This larger dataset was then used to retrain the forecasting model.

```
augmented_data = pd.concat([data, synthetic_data])
augmented_data = augmented_data.sort_values('ds')

augmented_data.head()
```

	unique_id	ds	y	hour	day_of_week	lag_1	rolling_mean_3
97603	sensor_1	2018-07-28 07:06:00	31.000000	7	5	31.0	31.0
97603	sensor_1	2018-07-28 07:06:00	30.887005	7	5	31.0	31.0
97576	sensor_1	2018-07-28 07:07:00	30.423565	7	5	31.0	31.0
97577	sensor_1	2018-07-28 07:07:00	31.348208	7	5	31.0	31.0
97578	sensor_1	2018-07-28 07:07:00	31.088378	7	5	31.0	31.0

Retraining the Model

The augmented dataset was split into training and testing sets. A new Linear Regression model was then trained using the same engineered features as the original model.

```
augmented_data = pd.concat([data, synthetic_data])

train_size_aug = int(len(augmented_data)*0.8)
train_aug = augmented_data.iloc[:train_size_aug]
test_aug = augmented_data.iloc[train_size_aug:]

X_train_aug = train_aug[['hour','day_of_week','lag_1','rolling_mean_3']]
y_train_aug = train_aug['y']

# THESE ARE REQUIRED
X_test_aug = test_aug[['hour','day_of_week','lag_1','rolling_mean_3']]
y_test_aug = test_aug['y']

model_aug = LinearRegression()
model_aug.fit(X_train_aug, y_train_aug)
```

```
▾ LinearRegression
LinearRegression()
```

Evaluating the Augmented Model

After retraining the model with the augmented dataset, predictions were generated and evaluated using MAE and MSE. This helps determine whether synthetic data improved forecasting performance.

```
predictions_aug = model_aug.predict(X_test_aug)

mae_aug = mean_absolute_error(y_test_aug, predictions_aug)
mse_aug = mean_squared_error(y_test_aug, predictions_aug)

print("Augmented MAE:", mae_aug)
print("Augmented MSE:", mse_aug)
```

```
Augmented MAE: 1.1445516015647523
Augmented MSE: 4.689924611419362
```

Impact of Generative Modeling

After augmenting the dataset with synthetic data, the model was retrained and evaluated. The results showed how adding synthetic data can influence model performance.

Using synthetic data can improve model generalization by exposing the model to more varied patterns. In IIoT applications, this is helpful when real-world data is limited or when rare operating conditions are underrepresented.

Conclusion

This lab demonstrated how time-series forecasting can be applied to IIoT temperature data. The dataset was cleaned, transformed, and prepared for modeling.

Feature engineering techniques such as lag values, rolling averages, and time-based features helped improve the model's performance. The Linear Regression model was able to capture general trends in the data.

The use of synthetic data further improved the model's accuracy, showing how generative techniques can enhance forecasting results. Overall, this lab highlights the importance of data preparation, feature engineering, and evaluation in building effective IIoT systems.

Reflection

In this lab, we learned how to work with time-series data and apply forecasting techniques to real-world IIoT datasets. we gained a better understanding of how to clean data, create meaningful features, and evaluate model performance.

One of the most important things we learned was how feature engineering improves predictions. Creating features such as lag values and rolling averages helped the model better understand patterns in the data.

We also learned how synthetic data can be used to improve model performance, especially when real-world data is limited. Overall, this lab helped me understand the full process of building and evaluating a forecasting model.

References

Kaggle IoT Temperature Forecasting Dataset
https://www.kaggle.com/code/koheimuramatsu/iot-temperature-forecasting

Scikit-learn Documentation
https://scikit-learn.org

Matplotlib Documentation
https://matplotlib.org

ITAI 3377 Course Materials